



Rapport de projet: Développement d'une Application de Surveillances des Marchés de Cryptomonnaies

1- Introduction

Ce rapport décrit le processus de développement d'une application web de suivi de cryptomonnaies (**La cryptomonnaie de l'avenir**), les fonctionnalités livrées, les techniques utilisées, ainsi que les leçons apprises.

2- Contexte et objectifs

Le projet consiste à développer une application qui permet aux utilisateurs de suivre l'évolution des cryptomonnaies en temps réel, de créer des alertes personnalisées, et utiliser un portefeuille virtuel pour simuler des investissements (pour des utilisateurs premiums).

Alors au début du projet j'ai commencé par mettre en place d'abord, un **backlog produit** (des **sprints** avec des **users stories**). Avant de reporter le backlog sur **Jira**. Puis j'ai écrit le cahier des charges. Ensuite j'ai étudié minutieusement les attentes du client, de comprendre le sujet, mettre en place toute l'architecture primordiale pour l'aboutissement du projet. Enfin commencer à rédiger du code.

3- Méthodologie Agile

Approche adoptive

Pour gérer efficacement ce projet en étant seul sur ce projet, j'ai adopté une méthodologie Agile basée sur **Scrum**, en adaptant certains aspects pour convenir au timing (c'est-à-dire les cours, les contraintes de temps, le contexte individuel). L'outil **Jira** permet de suivre les tâches, créer le backlog et organiser les sprints.

Création du backlog produit

J'ai commencé par créer un backlog produit détaillé contenant toutes les user stories. Chaque user story représentait une fonctionnalité spécifique de l'application. Les users stories ont été priorisés selon leur importance et leur impact sur le projet.

Exemples de users stories:

- Implémenter des endpoints **REST** pour exposer les données collectées.
- Mettre en place des tests unitaires pour valider les comportements critiques des interfaces.

- Vérifier que l'API est RESTful et respecte les standards pour une intégration fluide avec le frontend.

Gestion des sprints

J'ai découpé le projet en sprints de deux semaines. Chaque sprint avait des objectifs clairs et atteignables. Et alors le backlog de sprint provient du backlog produit en fonction des priorités et de la charge de travail estimée.

Exemple d'organisation de sprints:

- spring 3: Développement console et tests
- spring 4: Développement backend

Gestion des retards

Il est arrivé que certaines tâches prennent plus de temps que prévu, notamment lors du sprint 5 (Développement du frontend). Lorsque je ne respecte pas les délais d'un sprint, je réévaluerai les priorités et reporterai certaines tâches non critiques au sprint suivant.

Leçons apprises sur la méthode agile

J'ai appris à mieux prioriser les tâches pour maximiser la valeur livrée à chaque sprint. Ensuite, en travaillant seul sur le projet, j'ai compris l'importance d'une grande flexibilité et d'une rapidité face aux imprévus. Le fait d'utiliser Jira m'a également aidé à structurer mon travail et avoir une visibilité constante sur l'état d'avancement. Enfin j'effectue des rétrospectives rapides pour identifier ce qui fonctionnait bien et ce qui doit être amélioré (une amélioration continue).

Références (Preuves)

Sur le projet, vous trouverez ci-joint le document du backlog produit en pdf et des captures d'écrans provenant de mon compte Jira (sur le dépôt sur ametice et github).

4- Développement Technique

Une explication plus détaillée peut être consultée sur la documentation technique du projet

Processus développement

- technologies utilisés: backend (**spring-boot**), frontend (**React js**), base de données (**SQLite3**)
- Architecture: L'application est basée sur une architecture **RESTFUL**, avec un backend exposant des **API** sécurisées par token.
- Sécurité: Les pages utilisateurs sont protégées par un système d'authentification basé sur **JWT** de Spring-boot.

Tests

- Tests unitaires: des tests unitaires ont été écrits pour tous les fonctionnalités du backend en utilisant **JUnit** et les **Mocks**
- Couverture de tests: un rapport de couverture de test a été généré à l'aide de **sonarqube**, avec un taux de couverture atteint de **82%**. (Ci-joint une image)
- Tests de performances: **K6** a été utilisé pour simuler un grand nombre de requêtes simultanées et vérifier la robustesse du backend.
- Tests de sécurité: **OWASP ZAP** a été utilisé pour identifier les failles de sécurité potentielles.

5- DevOps

J'ai pu créer l'image du backend et celle du frontend. Et les tâches que je n'ai pu accomplir c'est la mise en place de l'intégration continue, ainsi des pipelines. Et pour finir je n'ai pas également le déploiement avec kubernetes et le monitoring (**Grafana** et **Prometheus**).

6- Livrables

- Code source versionnée sur Github
- Documentation technique et utilisateur
- Tableau de bord Agile (Jira, backlog produit)
- Rapport de tests (unitaires, performance, sécurité)
- Image du backend et du frontend
- Documentation du code du backend (Javadoc)

7- Non-Livrables

- Intégration continue
- Pipelines CI/CD
- Déploiement avec Kubernetes
- Monitoring (Grafana et Prometheus)

Conclusion

Ce projet m'a permis d'approfondir mes compétences en développement d'application web, sur les tests (comment il est important de tester toutes les fonctionnalités surtout les priorités afin de simuler le comportement du logiciel), par contre le plus important que j'ai appris, c'est la gestion de projet agile. Travailler seul sur un projet complet m'a permis de mieux organiser mon temps, de prioriser les tâches, et à être plus rigoureux dans le suivi des objectifs. La méthode agile est très efficace même pour une personne sur un projet.

Je remercie le professeur pour cette opportunité d'apprentissage.

Mouhamadou Ahibou DIALLO

ILSEN CLA

15/01/2025 sur Avignon